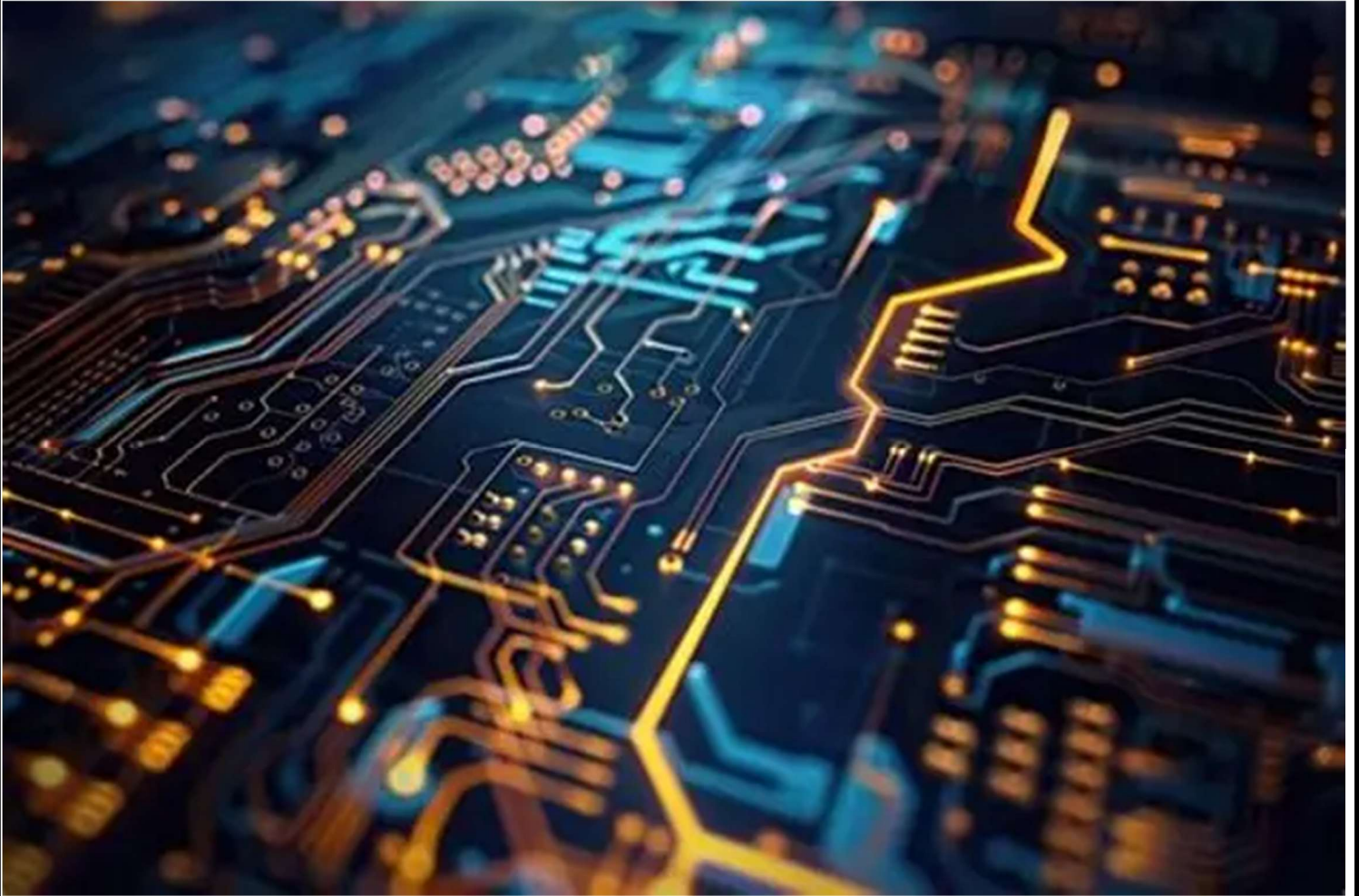


lab: Stream Parity Generation



Name: Ahmed Wael Sadek El
Sawy

lab: Stream Parity Generation

Stream Parity Generation

stream_parity_gen.v

```
module stream_parity_gen (  
    input  wire clk,  
    input  wire reset,  
    input  wire serial_in,  
    output wire parity_out,  
    output wire valid  
);  
  
    reg [7:0] shift_reg;  
    reg [3:0] bit_count;  
  
    function reg calc_even_parity;  
        input [7:0] data;  
        integer i;  
        reg p;  
        begin  
            p = 1'b0;  
            for (i = 0; i < 8; i = i + 1) begin  
                p = p ^ data[i];  
            end  
            calc_even_parity = p;  
        end  
    endfunction  
  
    always @(posedge clk) begin  
        if (reset) begin  
            shift_reg <= 8'b0;  
            bit_count <= 4'b0;  
        end else begin  
            shift_reg <= {shift_reg[6:0], serial_in};  
  
            if (bit_count == 4'd7) begin  
                bit_count <= 4'd1;  
            end else begin  
                bit_count <= bit_count + 1'b1;  
            end  
        end  
    end  
end
```

lab: Stream Parity Generation

```
assign valid = (bit_count == 4'd7);
assign parity_out = calc_even_parity(shift_reg);

endmodule
```

tb_stream_parity_gen.v

```
module tb_stream_parity_gen;

    reg clk;
    reg reset;
    reg serial_in;
    wire parity_out;
    wire valid;

    integer i;
    reg [7:0] test_data;

    stream_parity_gen DUT (
        .clk(clk),
        .reset(reset),
        .serial_in(serial_in),
        .parity_out(parity_out),
        .valid(valid)
    );

    // 50MHz clock generation (20ns period)
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    initial begin

        reset = 1;
        serial_in = 0;
        @(negedge clk);
        reset = 0;

        test_data = 8'b1011_0010;

        for (i = 7; i >= 0; i = i - 1) begin
            serial_in = test_data[i];
            @(negedge clk);
        end
    end
endmodule
```

lab: Stream Parity Generation

```
end

@(negedge clk);

test_data = 8'b1111_0010;
for (i = 7; i >= 0; i = i - 1) begin
    serial_in = test_data[i];
    @(negedge clk);
end

@(negedge clk);

$display("Simulation Complete.");
$stop;

end
endmodule
```

(Exhaustive self checking testbench)

```
module tb_stream_parity_gen;

    reg clk;
    reg reset;
    reg serial_in;
    wire parity_out;
    wire valid;

    integer errors = 0;

    stream_parity_gen DUT (
        .clk(clk),
        .reset(reset),
        .serial_in(serial_in),
        .parity_out(parity_out),
        .valid(valid)
    );

    // 50MHz clock generation (20ns period)
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end
end
```

lab: Stream Parity Generation

```
// Exhaustive Testing Task(self checking)

task run_exhaustive_test;
    integer val;
    integer bit_idx;
    reg [7:0] current_byte;
    reg expected_parity;
    begin
        $display("--- Starting Exhaustive Test (0 to 255) ---");

        // Outer loop: iterate through all possible 8-bit values (00 to FF)
        for (val = 0; val < 256; val = val + 1) begin
            current_byte = val;

            expected_parity = ^current_byte;

            // Inner loop: Serialize the 8 bits (MSB first)
            for (bit_idx = 7; bit_idx >= 0; bit_idx = bit_idx - 1) begin
                serial_in = current_byte[bit_idx];
                @(negedge clk);
            end

            if (parity_out != expected_parity) begin
                $display("ERROR: Parity mismatch for data %b. Expected %b,
got %b",
                        current_byte, expected_parity, parity_out);
                errors = errors + 1;
            end
        end
    end
endtask

// Main Simulation Block

initial begin

    reset = 1;
    serial_in = 0;
    @(negedge clk);
    reset = 0;

    run_exhaustive_test();
```

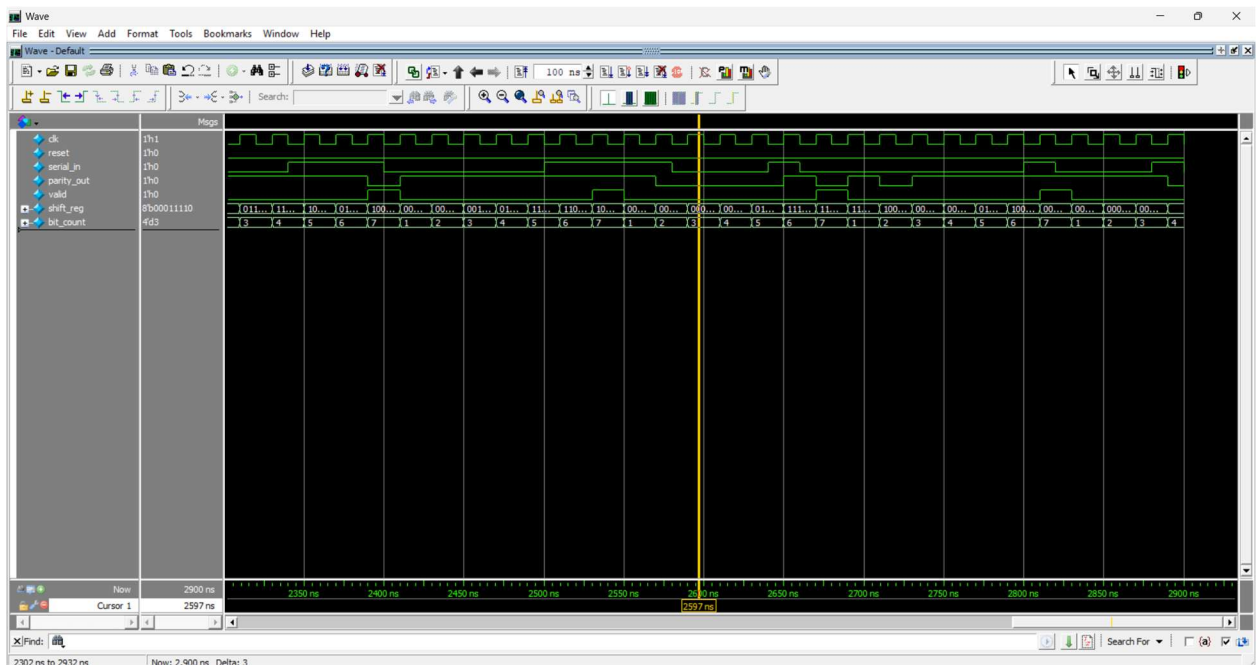
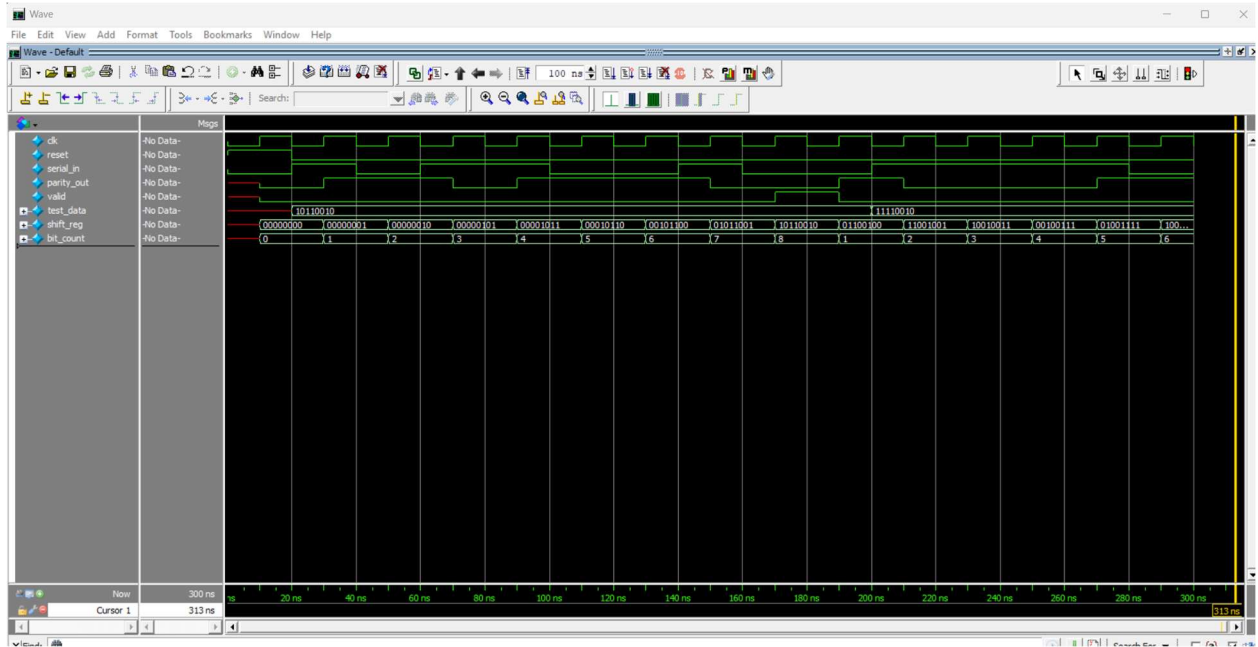
lab: Stream Parity Generation

```
if (errors == 0)
    $display("SUCCESS: All 256 combinations passed successfully.");
else
    $display("FAILED: System finished with %0d errors.", errors);

$stop;

end
endmodule
```

Testing stream parity generator



lab: Stream Parity Generation